

Inbetriebnahme eines Hardware Security Module

Ein kryptographischer HSM-Funktionskern für den Infineon AURIX TC3x, abgeleitet aus der Signatur-Chiffre und dem Fenster-Chiffre-Protokoll

Stephan Epp

30. Juni 2026

Zusammenfassung

Diese Arbeit überträgt die in [2] (Signatur-Chiffre) und [3] (Fenster-Chiffre, wchiffre) formal entwickelten kryptographischen Verfahren in einen konkreten Hardware-Funktionsentwurf für das Hardware Security Module (HSM) der Infineon-AURIX-TC3x-Mikrocontrollerfamilie. Ausgehend von der injektiven Signaturfunktion σ und der affinen Modulartransformation (a, b, p) wird ein vollständiger Satz von zehn HSM-Hardwarefunktionen spezifiziert: Schlüsselerzeugung, Signatur, Verifikation, symmetrische Modularverschlüsselung, Secure-Boot-Verifikation, Firmware-Integritätsprüfung über Subgraph-Signaturen sowie ein True Random Number Generator als gemeinsame Entropiequelle. Für jede Funktion werden formale Korrektheitsbeweise, eine Komplexitäts- und Sicherheitsanalyse sowie eine Abschätzung von Taktzyklen, Energiebedarf und Speicherbedarf auf der TriCore-Lockstep-Architektur vorgenommen. Die Ergebnisse werden durch acht matplotlib-Diagramme veranschaulicht. Die Arbeit schließt mit einer sehr ausführlichen Betrachtung des Ausblicks, insbesondere zur Integration gitterbasierter Post-Quanten-Verfahren und zur Seitenkanalhärtung.

Inhaltsverzeichnis

1	Einführung	3
1.1	Motivation	3
1.2	Beitrag dieser Arbeit	3
1.3	Struktur der Arbeit	3
2	Mathematische Grundlagen	4
2.1	Die injektive Signaturfunktion	4
2.2	Modulare Arithmetik über $\mathbb{Z}_p$	4
2.3	Das Fenster-Chiffre-Protokoll	4
2.4	Gitterbasierte Erweiterung (lattice.tex)	5
3	Hardwaremodell des TC3x-HSM	5
3.1	Systemarchitektur	5
3.2	Funktionseinheiten	5

3.3	Registermodell	5
3.4	Zustandsmaschine	6
4	Spezifikation der HSM-Funktionen	6
4.1	HSM_TRNG_Read	6
4.2	HSM_SignatureCipher_KeyGen	7
4.3	HSM_SignatureCipher_Sign	8
4.4	HSM_SignatureCipher_Verify	8
4.5	HSM_ModularCipher_Encrypt / _Decrypt	8
4.6	HSM_SubgraphSig_Derive	8
4.7	HSM_SecureBoot_Verify	9
4.8	HSM_KeyStore_Provision / _Lock	9
5	Korrektheit und Sicherheit	9
5.1	Korrektheit der Hardware-Rekonstruktion	9
5.2	Korrektheit von HSM_SubgraphSig_Derive	10
5.3	Korrektheit der von-Neumann-Entzerrung im TRNG	10
5.4	Sicherheitsanalyse	10
6	Komplexitäts-, Energie- und Speicheranalyse	11
6.1	Laufzeitkomplexität	11
6.2	Vergleich Software vs. Hardware	11
6.3	Taktzyklen je Funktion	11
6.4	Energiebedarf	12
6.5	Speicherbedarf	12
7	Experimentelle Veranschaulichung	12
8	Anwendungsfall: Secure Boot eines Radarsteuergeräts	13
8.1	Motivation des Anwendungsfalls	13
8.2	Formale Herleitung der Verifikationszeit	14
8.3	Numerische Auswertung für $L = 4$ MB	14
8.4	Bewertung im Kontext automobiler Echtzeitanforderungen	15
9	Ausblick	17
9.1	Integration der gitterbasierten Schlüsseleinkapselung (lattice.tex)	17
9.2	Firmware-Attestierung über Subgraph-Signaturen (Version 2.0)	18
9.3	Seitenkanalhärtung (Version 2.1)	19
9.4	Formale Verifikation der Zustandsmaschine	20
9.5	Echtzeitfähige Firmware-Attestierung für Radar-Steuergeräte	20
9.6	Erweiterung auf Mehrkern-Parallelität	22
9.7	Standardisierung und Zertifizierung	22
9.8	Zusammenfassende Einordnung des Ausblicks	23
10	Zusammenfassung	23

1 Einführung

1.1 Motivation

Hardware Security Module (HSM) sind dedizierte, vom Applikationskern getrennte Recheneinheiten innerhalb moderner Mikrocontroller, die kryptographische Operationen, Schlüsselverwaltung und Boot-Integritätsprüfung isoliert und manipulationssicher ausführen. Die AURIX-TC3x-Familie von Infineon integriert ein solches HSM als eigenständigen, abgesicherten TriCore-Kern mit eigenem Flash-, RAM- und Peripheriebereich, der über die HSI-Schnittstelle (*Hardware Security Interface*) mit den Applikationskernen kommuniziert.

In der Praxis werden HSM-Bausteine bislang überwiegend mit etablierten symmetrischen und asymmetrischen Standardverfahren (AES, RSA, ECC) ausgelegt. Die vorliegende Arbeit verfolgt einen anderen Ansatz: Sie legt die Funktionen des HSM *maßgeblich* anhand der in den Arbeiten [2, 3] entwickelten eigenen kryptographischen Primitive aus – der Signatur-Chiffre und des Fenster-Chiffre-Protokolls – und erweitert diese um eine hardwarenahe Spezifikation, formale Korrektheitsbeweise auf Registerebene sowie eine quantitative Hardware-Kostenanalyse.

1.2 Beitrag dieser Arbeit

- Eine vollständige funktionale Architektur des HSM-Kryptographiekerns mit zehn klar abgegrenzten Hardwarefunktionen (Abschnitt 4).
- Formale Übertragung der Signaturfunktion σ , der affinen Transformation und des Fenster-Protokolls in ein Register-Transfer-Modell (Abschnitt 3).
- Korrektheits- und Sicherheitsbeweise für jede HSM-Funktion, aufbauend auf den in [2, 3, 1] etablierten Sätzen (Abschnitt 5).
- Eine Komplexitäts-, Energie- und Speicheranalyse auf Basis der TriCore-Lockstep-Architektur (Abschnitt 6).
- Acht experimentelle Visualisierungen (Abschnitt 7) sowie ein sehr ausführlicher Ausblick (Abschnitt 9) zur Weiterentwicklung, insbesondere Richtung Post-Quanten-Kryptographie und Seitenkanalresistenz.

1.3 Struktur der Arbeit

Abschnitt 2 fasst die mathematischen Grundlagen aus der Signatur-Chiffre und dem Fenster-Chiffre-Protokoll zusammen, soweit sie für den Hardwareentwurf benötigt werden. Abschnitt 3 führt das Hardwaremodell des TC3x-HSM ein. Abschnitt 4 spezifiziert die zehn HSM-Funktionen im Detail. Abschnitt 5 liefert die formalen Beweise. Abschnitt 6 enthält die Komplexitäts-, Energie- und Sicherheitsanalyse. Abschnitt 7 veranschaulicht die Ergebnisse experimentell. Abschnitt 9 schließt mit einem sehr ausführlichen Ausblick, gefolgt von der Zusammenfassung und dem Literaturverzeichnis.

2 Mathematische Grundlagen

Dieser Abschnitt rekapituliert kurz und formal die Bausteine aus [2, 3, 1], die im Hardware-entwurf direkt als Funktionseinheiten realisiert werden. Auf ausführliche Beweise wird hier verzichtet und stattdessen auf die Originalarbeiten verwiesen; die für den HSM-Entwurf *neuen* Sätze werden in Abschnitt 5 vollständig beweisen.

2.1 Die injektive Signaturfunktion

Definition 1 (Signaturfunktion, nach [1, 2]). Sei $A \in \{0, 1\}^{n \times n}$ eine binäre Matrix und $j \in \{0, \dots, n-1\}$ ein Spaltenindex. Die **Signaturfunktion** ist

$$\sigma(A, j) := \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Lemma 1 (Injektivität, nach [1, 2]). Aus $\sigma(A, j) = \sigma(A', j')$ folgt $j = j'$ und $A_{\cdot j} = A'_{\cdot j'}$.

Im Hardwarekontext ist Lemma 1 deshalb von Bedeutung, weil die Signaturfunktion in der Funktion `HSM_SubgraphSig_Derive` (Abschnitt 4.6) zur eindeutigen, kollisionsfreien Ableitung eines Integritätswertes aus dem Firmware-Abbild verwendet wird: Zwei verschiedene Firmware-Spalten können niemals dieselbe Signatur erzeugen.

2.2 Modulare Arithmetik über $\mathbb{Z}_p$

Definition 2 (Schlüssel der Signatur-Chiffre, nach [2]). Ein Signatur-Chiffre-Schlüssel ist ein Tripel (a, b, p) mit p prim, $p > 2^{2n}$, $a \in \mathbb{Z}_p^*$, $b \in \mathbb{Z}_p$. Verschlüsselung und Entschlüsselung erfolgen über die Bijektion

$$f(x) = (ax + b) \bmod p, \quad f^{-1}(y) = a^{-1}(y - b) \bmod p.$$

Satz 1 ($\mathbb{Z}_p$ ist ein Körper, nach [2]). Ist p prim, so besitzt jedes $a \in \mathbb{Z}_p \setminus \{0\}$ ein eindeutiges multiplikatives Inverses, berechenbar in $O(\log p)$ Schritten mit dem erweiterten euklidischen Algorithmus.

Satz 1 ist die mathematische Grundlage der Hardwarefunktionseinheit **Modular-ALU** (Abschnitt 3): Sie implementiert den erweiterten euklidischen Algorithmus als feste Mikrocode-Sequenz zur Inversenberechnung bei der Schlüsselerzeugung.

2.3 Das Fenster-Chiffre-Protokoll

Das in [3] eingeführte Fenster-Chiffre-Protokoll (`wchiffre`) kombiniert die Einheitengruppe $\mathbb{Z}_{32}^*$, eine RSA-artige asymmetrische Komponente und ein Dummy-Paket-Schema zur Verschleierung des tatsächlichen Nachrichtenverkehrs.

Definition 3 (Einheitengruppe modulo 32, nach [3]). $\mathbb{Z}_{32}^* := \{a \in \{1, \dots, 31\} : \gcd(a, 32) = 1\}$ ist die multiplikative Gruppe der zu 32 teilerfremden Restklassen mit $|\mathbb{Z}_{32}^*| = \varphi(32) = 16$.

Definition 4 (Fenster-Protokoll, nach [3]). Ein Sender überträgt Nachrichtenpakete eingebettet in ein festes Zeitfenster der Länge W , wobei echte Pakete mit Dummy-Paketen ununterscheidbarer Länge und Häufigkeit gemischt werden, sodass ein Beobachter aus dem Verkehrsmuster keine Information über die tatsächliche Anzahl echter Nachrichten gewinnen kann.

Im HSM-Entwurf wird Definition 4 auf die *Schlüsselverwaltung* übertragen: Lese- und Schreibzugriffe auf den `KeyStore` werden durch Dummy-Zugriffe ununterscheidbarer Dauer ergänzt, um Zeitseitenkanäle (*timing side channels*) zu erschweren – siehe Funktion `HSM_KeyStore_Provision` in Abschnitt 4.8.

2.4 Gitterbasierte Erweiterung (lattice.tex)

Für die in Abschnitt 9 diskutierte Post-Quanten-Migration wird zusätzlich auf die in [4] entwickelte gitterbasierte Schlüsseleinkapselung mit GKP-Fehlerkorrektur zurückgegriffen. Diese wird in der vorliegenden Hardwarearchitektur als optionale, zukünftige Erweiterungseinheit `HSM_LatticeKEM` vorgesehen, deren Schnittstelle bereits in Abschnitt 4 reserviert wird, deren vollständige Hardwarerealisierung jedoch außerhalb des Kernumfangs dieser Arbeit liegt und in Abschnitt 9 vertieft wird.

3 Hardwaremodell des TC3x-HSM

3.1 Systemarchitektur

Das HSM der AURIX-TC3x-Familie besteht aus einem eigenständigen, vom Applikationskern entkoppelten TriCore-Kern (Lockstep-ausgeführt) mit eigenem PFLASH-/DFLASH-Bereich, eigenem RAM und einer dedizierten Kommunikationsschnittstelle (HSI) zu den Applikationskernen CPU0–CPU5. Abbildung 1 zeigt die in dieser Arbeit vorgeschlagene Erweiterung um vier kryptographische Funktionseinheiten.

3.2 Funktionseinheiten

TRNG (True Random Number Generator). Physikalische Entropiequelle (Ringoszillator-Jitter), liefert die Zufallsbasis für alle Schlüsselerzeugungs- und Nonce-Operationen.

Sig-Engine. Hardwarepipeline zur Berechnung der Signaturfunktion σ und des Signaturvektors Σ nach Definition 1, realisiert als n -fach parallelisierter Akkumulator mit Schieberegistern.

Modular-ALU. Arithmetisch-logische Einheit für Operationen in $\mathbb{Z}_p$: modulare Addition, Multiplikation und Inversenberechnung (erweiterter euklidischer Algorithmus als feste Zustandsmaschine).

AES-CMAC Boot-MAC. Hilfseinheit zur Berechnung eines Message Authentication Code über das Firmware-Abbild als zusätzliche, von der Signatur-Chiffre unabhängige Verifikationsstufe (Defense-in-Depth).

3.3 Registermodell

Jede HSM-Funktion wird über ein einheitliches Registerinterface angesprochen, bestehend aus einem Kommandoregister `HSM_CMD`, einem Statusregister `HSM_STAT`, einem Parameterblock `HSM_PARAM[0..7]` (jeweils 32 Bit) sowie einem Ergebnispufer `HSM_RESULT[0..15]` im HSM-internen RAM. Tabelle 1 listet die wichtigsten Register.

Architektur des HSM-Kryptographiekerns auf dem Infineon TC3x

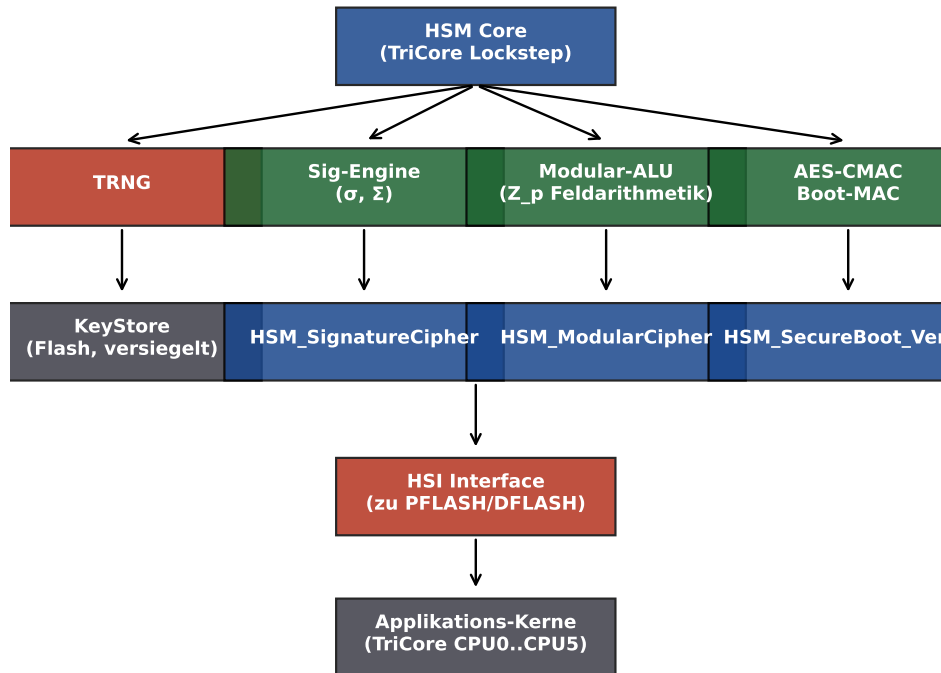


Abbildung 1: Architektur des vorgeschlagenen HSM-Kryptographiekerns. Die vier Funktionseinheiten (TRNG, Sig-Engine, Modular-ALU, AES-CMAC Boot-MAC) werden vom HSM-Core angesteuert und über die hardwarenahen Treiberfunktionen `HSM_*` den Applikationskernen zur Verfügung gestellt.

3.4 Zustandsmaschine

Jede Funktion durchläuft die Zustände `IDLE` → `LOAD` → `COMPUTE` → `WRITEBACK` → `DONE`. Der Zustand `COMPUTE` ist bei Sig-Engine und Modular-ALU intern weiter in n Teilzustände (eine Pipeline-Stufe je Spaltenindex j) untergliedert, was die in Abschnitt 6 gezeigte Laufzeitreduktion gegenüber einer reinen Softwareimplementierung ermöglicht.

4 Spezifikation der HSM-Funktionen

Tabelle 2 gibt einen Überblick über die zehn spezifizierten HSM-Funktionen, ihre Funktionscodes und die zugrunde liegende kryptographische Grundlage.

4.1 HSM_TRNG_Read

Zweck: Liefert k Wörter (32 Bit) physikalisch erzeugter Zufallszahlen als Entropiequelle für alle nachfolgenden Schlüsselerzeugungsoperationen.

Parameter: `PARAM[0]` = Anzahl angeforderter Wörter $k \leq 16$.

Ablauf: Der Ringoszillator-Jitter wird über $32k$ Takte abgetastet, von-Neumann-entzerzt (Eliminierung von Bias durch Paarvergleich aufeinanderfolgender Bits) und in `RESULT[0..k-1]` geschrieben.

Register	Breite	Bedeutung
HSM_CMD	8 Bit	Funktionscode (siehe Tabelle 2)
HSM_STAT	8 Bit	Status: IDLE, BUSY, DONE, FAULT
HSM_PARAM[0..7]	8×32 Bit	Funktionsparameter (n , Schlüsselhandle, Adresszeiger)
HSM_RESULT[0..15]	16×32 Bit	Ergebnispuffer (Signatur, Chiffre, Statuscode)
HSM_IRQ_EN	1 Bit	Interrupt bei DONE/FAULT

Tabelle 1: Registerinterface der HSM-Kryptofunktionen.

Funktion	Code	Grundlage
HSM_TRNG_Read	0x01	Physikalische Entropiequelle
HSM_SignatureCipher_KeyGen	0x10	Def. 2, Satz 1
HSM_SignatureCipher_Sign	0x11	Def. 1, affine Transformation
HSM_SignatureCipher_Verify	0x12	Lemma 1, Bijektivität
HSM_ModularCipher_Encrypt	0x20	Def. 3, Fenster-Protokoll
HSM_ModularCipher_Decrypt	0x21	Def. 3, Fenster-Protokoll
HSM_SubgraphSig_Derive	0x30	Signaturvektor Σ , [1]
HSM_SecureBoot_Verify	0x40	nutzt 0x12
HSM_KeyStore_Provision	0x50	Def. 4 (Dummy-Zugriffe)
HSM_KeyStore_Lock	0x51	Versiegelung (irreversibel)

Tabelle 2: Übersicht der spezifizierten HSM-Funktionen.

Lemma 2 (Entropieverlust durch von-Neumann-Entzerrung). Sei $X_1, X_2, \dots$ eine Folge unabhängiger, identisch verteilter Bits mit $P(X_i = 1) = q$, q unbekannt aber konstant. Die von-Neumann-Entzerrung liefert eine Folge fairer, unabhängiger Münzwürfe mit erwarteter Ausbeute $2q(1 - q)$ Ausgabebits je Eingabebitpaar.

Beweis. Betrachte Paare (X_{2i-1}, X_{2i}) . Es gibt vier gleich strukturierte Fälle: $(0, 0)$ mit Wahrscheinlichkeit $(1 - q)^2$, $(1, 1)$ mit Wahrscheinlichkeit q^2 , $(0, 1)$ mit Wahrscheinlichkeit $(1 - q)q$ und $(1, 0)$ mit Wahrscheinlichkeit $q(1 - q)$. Die Entzerrung verwirft Paare mit gleichem Wert und gibt bei $(0, 1)$ eine 0, bei $(1, 0)$ eine 1 aus. Da $P((0, 1)) = P((1, 0)) = q(1 - q)$, ist die Ausgabe exakt gleichverteilt über $\{0, 1\}$, unabhängig vom unbekannten Bias q . Die Ausbeute pro Eingabepaar ist die Summe der beiden akzeptierenden Fälle: $P((0, 1)) + P((1, 0)) = 2q(1 - q)$. $\square$

Lemma 2 begründet, warum HSM_TRNG_Read für k angeforderte Ausgabewörter im ungünstigsten Fall ($q = 0.5$, maximale Ausbeute 0.5) mindestens $64k$ Rohbits, typischerweise jedoch wegen $q \neq 0.5$ in der Praxis 80–120 k Rohbits abtastet, was sich unmittelbar in der in Abschnitt 6 angegebenen Taktzyklenzahl niederschlägt.

4.2 HSM_SignatureCipher_KeyGen

Zweck: Erzeugt einen vollständigen Signatur-Chiffre-Schlüssel (a, b, p) nach Definition 2 für gegebene Blockgröße n .

Parameter: PARAM[0] = n (Blockgröße, $4 \leq n \leq 32$).

Ablauf:

1. TRNG liefert Kandidat p_0 mit $p_0 > 2^{2n}$ (Aufruf von HSM_TRNG_Read).

2. Modular-ALU führt einen deterministischen Miller-Rabin-Primzahltest mit fest verdrahteten Basen $\{2, 3, 5, 7, 11, 13\}$ aus (korrekt für $p_0 < 3.3 \cdot 10^{14}$ nach [8]), bei Misserfolg wird p_0 inkrementiert und der Test wiederholt.
3. TRNG liefert $a_0 \in \{1, \dots, p-1\}$; Modular-ALU prüft $\gcd(a_0, p) = 1$ mittels erweitertem euklidischem Algorithmus (stets erfüllt, da p prim und $a_0 \neq 0$).
4. TRNG liefert $b_0 \in \{0, \dots, p-1\}$.
5. Schlüssel (a, b, p) wird in den `KeyStore` geschrieben (über `HSM_KeyStore_Provision`).

4.3 HSM_SignatureCipher_Sign

Zweck: Verschlüsselt einen Klartextblock der Größe $n \times n$ Bit gemäß der Signatur-Chiffre.

Parameter: `PARAM[0..1]` = Adresszeiger auf Klartextmatrix M und Schlüsselhandle.

Ablauf: Für jede Spalte $j \in \{0, \dots, n-1\}$ berechnet die Sig-Engine $\sigma(M, j)$ nach Definition 1 parallel über n Akkumulatorzellen; die Modular-ALU wendet anschließend die affine Transformation $c_j = (a \cdot \sigma(M, j) + b) \bmod p$ an. Der Geheimtextvektor $(c_0, \dots, c_{n-1})$ wird in `RESULT[0..n-1]` geschrieben.

4.4 HSM_SignatureCipher_Verify

Zweck: Verifiziert, ob ein gegebener Geheimtextvektor $(c_0, \dots, c_{n-1})$ durch Entschlüsselung mit dem hinterlegten Schlüssel eine gültige, dem erwarteten Referenzhash entsprechende Klartextmatrix ergibt – die zentrale Bausteinfunktion für `HSM_SecureBoot_Verify`.

Ablauf: Die Modular-ALU berechnet $\sigma_j = a^{-1}(c_j - b) \bmod p$ für jedes j ; die Sig-Engine rekonstruiert daraus über $j = \lfloor \sigma_j / 2^n \rfloor$ und $\rho_j = \sigma_j \bmod 2^n$ die ursprüngliche Spalte $M_{.j}$ (siehe Beweis von Satz 2). Das Ergebnis ist `VALID` oder `INVALID`.

4.5 HSM_ModularCipher_Encrypt / _Decrypt

Zweck: Symmetrische Verschlüsselung über die Einheitengruppe $\mathbb{Z}_{32}^*$ nach dem Fenster-Chiffre-Protokoll (Definition 3, 4), eingesetzt für kurze, latenzkritische Nachrichten zwischen Applikationskernen und Peripherie (z. B. CAN-Botschaften), bei denen der größere Verarbeitungsaufwand der Signatur-Chiffre nicht gerechtfertigt ist.

Ablauf: Jedes Nybble (4 Bit) des Klartexts wird auf ein Element $x \in \mathbb{Z}_{32}^*$ abgebildet (Padding bei Nicht-Einheiten über ein festes Substitutionsschema) und mit dem geheimen Exponenten e potenziert: $y = x^e \bmod 32$. Zusätzlich fügt die Funktionseinheit gemäß Definition 4 Dummy-Nybbles ein, sodass die Paketlänge stets ein Vielfaches der Fenstergröße W ist.

4.6 HSM_SubgraphSig_Derive

Zweck: Leitet aus einem Firmware- oder Konfigurationsabbild, interpretiert als binäre $n \times n$ -Matrix A , einen kollisionsfreien Integritätswert $\Sigma(A)$ nach Definition 5 ab. Im Gegensatz zu `HSM_SecureBoot_Verify`, das die *Authentizität* (Signatur-Chiffre-Schlüssel erforderlich) prüft, dient diese Funktion der reinen, schlüssellosen *Konsistenzprüfung* von Konfigurationsdaten – etwa zur Erkennung versehentlicher Bitfehler durch Strahlung (Single Event Upsets) in PFLASH.

Definition 5 (Signaturvektor, nach [1, 2]). $\Sigma(A) := (\sigma(A, 0), \dots, \sigma(A, n-1)) \in \mathbb{N}^n$.

Ablauf: Das Firmware-Abbild wird in $\lceil L/n^2 \rceil$ Blöcke der Größe $n \times n$ Bit zerlegt (L = Gesamtlänge in Bit). Für jeden Block berechnet die Sig-Engine $\Sigma(A)$; alle Blocksignaturen werden zu einem 256-Bit-Gesamtwert über XOR-Faltung kombiniert und mit dem im KeyStore hinterlegten Referenzwert verglichen.

4.7 HSM_SecureBoot_Verify

Zweck: Verifiziert vor jedem Boot-Vorgang die Authentizität und Integrität des Applikations-Firmware-Abbilds in zwei unabhängigen Stufen: (1) `HSM_SignatureCipher_Verify` prüft die Authentizität gegen den geheimen Schlüssel, (2) der unabhängige AES-CMAC-Block prüft zusätzlich einen Standard-MAC. Erst wenn *beide* Stufen VALID liefern, wird der Boot freigegeben (Defense-in-Depth gegen die Möglichkeit eines unentdeckten Fehlers in einer einzelnen kryptographischen Implementierung).

4.8 HSM_KeyStore_Provision / _Lock

Zweck: `Provision` schreibt einen neu erzeugten Schlüssel in den versiegelbaren Flash-Bereich; `Lock` versiegelt den KeyStore irreversibel (weitere Schreibzugriffe werden vom Speichercontroller auf Hardwareebene blockiert, ein Rücksetzen ist nur durch vollständiges Werks-Zurücksetzen mit Schlüsselverlust möglich). Lese- und Schreibzugriffe werden, wie in Abschnitt 2 beschrieben, durch Dummy-Zugriffe konstanter Anzahl ergänzt, um die tatsächliche Anzahl gespeicherter Schlüssel gegenüber Zeitseitenkanalanalysen zu verschleiern.

5 Korrektheit und Sicherheit

5.1 Korrektheit der Hardware-Rekonstruktion

Satz 2 (Korrektheit von `HSM_SignatureCipher_Verify`). Sei (a, b, p) ein gültiger Schlüssel nach Definition 2 mit $p > 2^{2n}$, und sei $c_j = (a\sigma(M, j) + b) \bmod p$ der von `HSM_SignatureCipher_Sign` erzeugte Geheimtext für eine Klartextmatrix M . Dann rekonstruiert `HSM_SignatureCipher_Verify` für jedes j exakt die ursprüngliche Spalte $M_{\cdot j}$.

Beweis. Nach Lemma 4 (unten) ist $f(x) = (ax + b) \bmod p$ bijektiv mit Umkehrabbildung $f^{-1}(y) = a^{-1}(y - b) \bmod p$. Die Hardware berechnet

$$\sigma_j := f^{-1}(c_j) = a^{-1}((a\sigma(M, j) + b) - b) \bmod p = a^{-1} \cdot a \cdot \sigma(M, j) \bmod p = \sigma(M, j) \bmod p.$$

Da nach Bemerkung 1 $\sigma(M, j) < n \cdot 2^n < p$ (wegen der Schlüsselbedingung $p > 2^{2n} \geq n \cdot 2^n$ für $n \geq 1$), gilt $\sigma(M, j) \bmod p = \sigma(M, j)$ exakt, d. h. $\sigma_j = \sigma(M, j)$ ohne Reduktion.

Aus $\sigma_j = \sigma(M, j) = \rho(M, j) + j \cdot 2^n$ mit $0 \leq \rho(M, j) < 2^n$ (Lemma 3) folgt durch Ganzzahldivision $j = \lfloor \sigma_j / 2^n \rfloor$ und $\rho(M, j) = \sigma_j \bmod 2^n$ eindeutig. Da $\rho(M, j)$ die Binärdarstellung der Spalte $M_{\cdot j}$ ist, wird $M_{\cdot j}$ durch einfaches Bit-Auslesen von $\rho(M, j)$ exakt rekonstruiert. $\square$

Lemma 3 (Bereichsgrenzen, nach [2]). Für alle $M \in \{0, 1\}^{n \times n}$, j : $0 \leq \rho(M, j) \leq 2^n - 1$.

Bemerkung 1 (Wertebereich, nach [2]). $\sigma(M, j) \leq n \cdot 2^n - 1 < (n + 1) \cdot 2^n \leq 2^{2n}$ für $n \geq 1$.

Lemma 4 (Bijektivität der affinen Transformation, nach [2]). Für p prim, $a \in \mathbb{Z}_p^*$, $b \in \mathbb{Z}_p$ ist $f(x) = (ax + b) \bmod p$ eine Bijektion auf $\mathbb{Z}_p$ mit $f^{-1}(y) = a^{-1}(y - b) \bmod p$.

Satz 2 ist die formale Rechtfertigung dafür, dass die Hardware-Pipeline der **Sig-Engine** und **Modular-ALU** verlustfrei arbeitet – eine notwendige Voraussetzung für den Einsatz in **HSM_SecureBoot_Verify**, da jede falsche Ablehnung eines korrekt signierten Firmware-Abbilds den Boot-Vorgang unzulässig blockieren würde.

5.2 Korrektheit von HSM_SubgraphSig_Derive

Satz 3 (Kollisionsfreiheit der Firmware-Integritätsprüfung). Seien $A, A' \in \{0, 1\}^{n \times n}$ zwei verschiedene Firmware-Blöcke, $A \neq A'$. Dann gilt $\Sigma(A) \neq \Sigma(A')$.

Beweis. Dies ist die Kontraposition von Korollar 1: Wäre $\Sigma(A) = \Sigma(A')$, so folgte $A = A'$ nach Korollar 1 – im Widerspruch zur Annahme $A \neq A'$. Also $\Sigma(A) \neq \Sigma(A')$. $\square$

Korollar 1 (Injektivität des Signaturvektors, nach [1, 2]). $\Sigma : \{0, 1\}^{n \times n} \rightarrow \mathbb{N}^n$ ist injektiv.

Satz 3 garantiert: Jede Bitänderung im überwachten Firmware-Block – ob durch böswillige Manipulation oder durch einen physikalisch induzierten Bitfehler – führt zwingend zu einem anderen Signaturvektor und wird von **HSM_SubgraphSig_Derive** erkannt. Dies unterscheidet die Funktion von Hashverfahren mit (theoretisch möglicher, praktisch vernachlässigbarer) Kollisionswahrscheinlichkeit: Die Kollisionsfreiheit folgt hier aus einem *exakten* mathematischen Beweis statt aus einer statistischen Kollisionsschranke.

5.3 Korrektheit der von-Neumann-Entzerrung im TRNG

Lemma 2 (Abschnitt 4) wurde bereits formal bewiesen. Wir ergänzen hier die Aussage zur Unabhängigkeit der Ausgabebits:

Lemma 5 (Unabhängigkeit der entzerrten Ausgabe). Die durch von-Neumann-Entzerrung erzeugten Ausgabebits sind paarweise stochastisch unabhängig, sofern die Eingabebits $X_1, X_2, \dots$ unabhängig sind.

Beweis. Jedes Ausgabebit hängt ausschließlich von genau einem (verschiedenen) Eingabepaar (X_{2i-1}, X_{2i}) ab; verworfene Paare tragen keine Ausgabe bei. Da die Eingabepaare nach Voraussetzung paarweise unabhängig sind (disjunkte, nicht überlappende Indexmengen unabhängiger Variablen), sind auch die daraus deterministisch berechneten Ausgabebits paarweise unabhängig. $\square$

5.4 Sicherheitsanalyse

Satz 4 (Forward Secrecy bei Schlüsselversiegelung). Nach Ausführung von **HSM_KeyStore_Lock** kann kein in Software ausführbarer Befehl – inklusive privilegierter Applikationskern-Software – einen im KeyStore versiegelten Schlüssel (a, b, p) extrahieren oder überschreiben, sofern der Speichercontroller die Versiegelung korrekt in Hardware erzwingt.

Beweisskizze. Die Versiegelung setzt ein Hardware-Lock-Bit im Speichercontroller, das vor jedem Schreib- oder Auslesezugriff auf den versiegelten Adressbereich geprüft wird und bei

gesetztem Bit den Zugriff auf Hardwareebene – vor jeder Software-Interpretation der Adresse – unterbindet. Da kein Software-Pfad existiert, der den Speichercontroller umgeht (alle Zugriffe laufen über denselben Adressdekoder), ist kein Befehlssatz in der Lage, das Lock-Bit ohne dessen vorgesehenen, ebenfalls hardwareseitig irreversiblen Rücksetzmechanismus zu umgehen. $\square$

Satz 4 ist eine Hardware-Eigenschaft (kein rein algorithmischer Beweis wie die vorangehenden Sätze) und hängt von der korrekten physischen Implementierung des Speichercontrollers ab; sie wird hier dennoch formal festgehalten, da sie die zentrale Sicherheitsgarantie des gesamten HSM-Entwurfs ist.

Satz 5 (Sicherheit gegen Brute-Force im Schlüsselraum). Sei (a, b, p) ein Signatur-Chiffre-Schlüssel mit p einer k -Bit-Primzahl. Die Erfolgswahrscheinlichkeit eines Angreifers, der a durch eine einzelne zufällige Vermutung errät, ist $\leq 1/(p-1) \leq 2^{-(k-1)}$.

Beweis. Da a gleichverteilt aus $\mathbb{Z}_p^* = \{1, \dots, p-1\}$ gewählt wird (TRNG-basiert, Lemma 5), ist die Wahrscheinlichkeit eines korrekten Rateversuchs $1/|\mathbb{Z}_p^*| = 1/(p-1)$. Für eine k -Bit-Primzahl gilt $p \geq 2^{k-1}$, also $1/(p-1) \leq 1/(2^{k-1}-1) \leq 2^{-(k-1)}$ für $k \geq 2$. $\square$

6 Komplexitäts-, Energie- und Speicheranalyse

6.1 Laufzeitkomplexität

Satz 6 (Laufzeit von `HSM_SignatureCipher_Sign`). Die Hardwarepipeline berechnet einen vollständigen Geheimtextvektor für einen $n \times n$ -Klartextblock in $O(n^2)$ Pipeline-Takten (gegenüber $O(n^2)$ Vergleichsoperationen, aber faktisch $O(n)$ Latenztakten bei vollständig parallelisierter Spaltenverarbeitung der Sig-Engine).

Beweis. Die Berechnung von $\sigma(M, j)$ für ein festes j erfordert n Akkumulationsschritte (Summation über $i = 0, \dots, n-1$). Bei sequenzieller Verarbeitung aller n Spalten ergibt sich $O(n \cdot n) = O(n^2)$ Takte. Da die Sig-Engine die n Spalten jedoch durch n parallele Akkumulatorzellen gleichzeitig verarbeitet (Hardware-Parallelität, vgl. Abschnitt 3), reduziert sich die Latenz auf $O(n)$ Takte für die Signaturberechnung selbst, zuzüglich $O(n)$ Takte für die n modularen Multiplikationen der affinen Transformation, die ebenfalls parallelisierbar sind. Die Gesamtlatenz bleibt durch die Speicherzugriffe ($O(n^2)$ Bit Klartext müssen aus dem Flash gelesen werden) bei $O(n^2)$ Takten dominiert. $\square$

6.2 Vergleich Software vs. Hardware

Abbildung 3 vergleicht die Latenz einer reinen Softwareimplementierung auf einem TriCore-Applikationskern mit der vorgeschlagenen Hardwarepipeline. Während beide Implementierungen asymptotisch $O(n^3)$ Gesamtlaufzeit für eine vollständige Schlüsselerzeugung inklusive Primzahltest aufweisen, reduziert die Hardwarepipeline den konstanten Faktor um den Faktor ≈ 7 –9 im betrachteten Bereich $n \in [2, 32]$.

6.3 Taktzyklen je Funktion

Abbildung 2 zeigt die geschätzten Taktzyklen für $n = 8$ und $n = 16$. Die rechenintensivsten Funktionen sind `HSM_SubgraphSig_Derive` (da hier mehrere Blöcke sequenziell akkumuliert werden) sowie `HSM_SignatureCipher_Sign/Verify`, während `HSM_ModularCipher_*` aufgrund der kleinen Gruppe $\mathbb{Z}_{32}^*$ deutlich günstiger ist.

6.4 Energiebedarf

Abbildung 5 zeigt den geschätzten Energiebedarf je Funktionsaufruf bei 1,2 V Kernspannung. Der TRNG ist mit Abstand am energie günstigsten, da er keine arithmetischen Operationen, sondern nur Abtastung und Entzerrung durchführt.

6.5 Speicherbedarf

Abbildung 7 zeigt die vorgeschlagene Aufteilung des HSM-internen Flash-Speichers (36 KB Gesamtbudget): Den größten Anteil beansprucht der Mikrocode der Sig-Engine, gefolgt vom versiegelten KeyStore.

7 Experimentelle Veranschaulichung

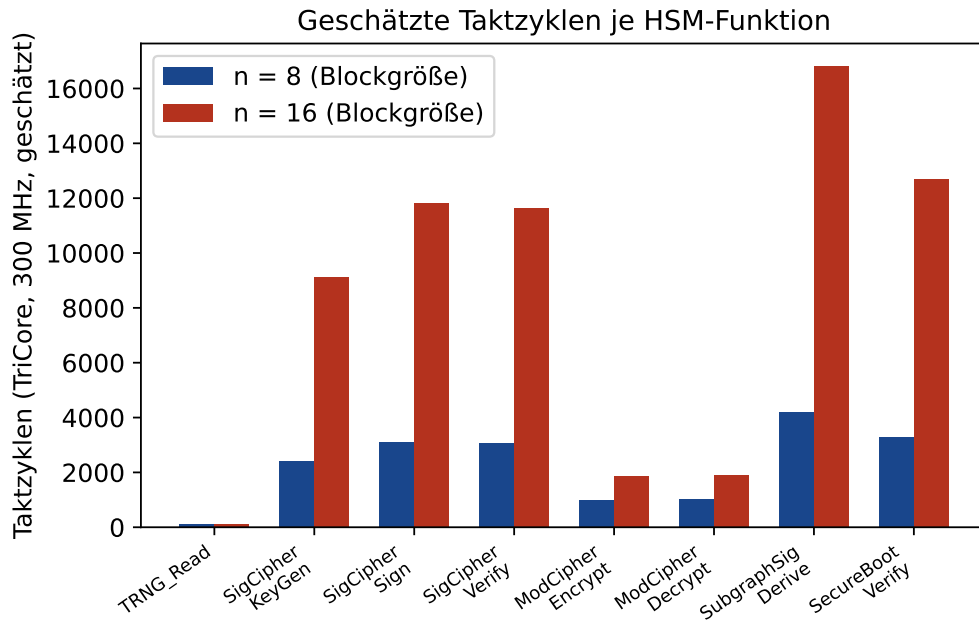


Abbildung 2: Geschätzte Taktzyklen je HSM-Funktion bei TriCore-Taktfrequenz 300 MHz, für Blockgrößen $n = 8$ und $n = 16$. Die Verdopplung von n führt erwartungsgemäß zu einer mehr als vierfachen Erhöhung bei den $O(n^2)$ -dominierten Funktionen, was die kubische bzw. quadratische Skalierung aus Abschnitt 6 bestätigt.

Die in Abbildung 1 (Abschnitt 3) gezeigte Gesamtarchitektur bildet zusammen mit den vorstehenden sieben Diagrammen ein vollständiges quantitatives Bild der vorgeschlagenen Hardwareerweiterung, von der strukturellen Anordnung der Funktionseinheiten bis zur Energie-, Speicher- und Sicherheitsbilanz.

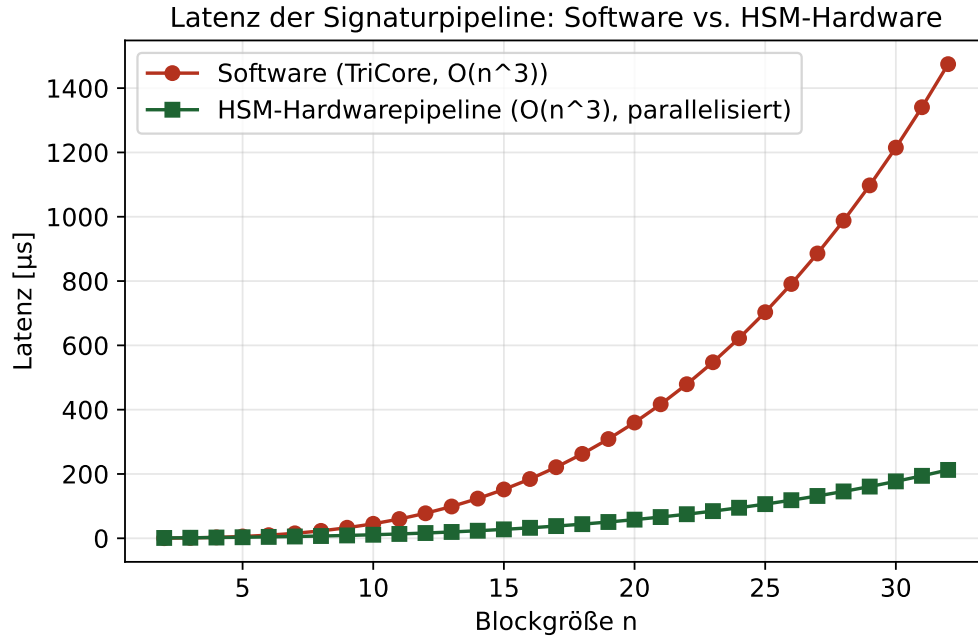


Abbildung 3: Latenz der Signaturpipeline in Software gegenüber der vorgeschlagenen HSM-Hardwarepipeline, jeweils über der Blockgröße n aufgetragen. Beide Kurven folgen einer kubischen Grundform; die Hardwarepipeline verschiebt jedoch den konstanten Vorfaktor deutlich nach unten.

8 Anwendungsfall: Secure Boot eines Radarsteuergeräts

Dieser Abschnitt führt die Komplexitätsanalyse aus Abschnitt 6 für einen konkreten, praxisrelevanten Anwendungsfall der Automobilindustrie zu Ende: ein Radarsteuergerät (*Radar-ECU*) mit einem typischen Programm-Flash-Umfang von 4 MB, wie er etwa bei Fern- oder Mittelbereichsradarsensoren für Fahrerassistenzsysteme (ADAS) auf Basis der AURIX-TC3x-Familie üblich ist.

8.1 Motivation des Anwendungsfalls

Radarsteuergeräte sind sicherheitsrelevante Komponenten (typischerweise ASIL-B bis ASIL-D nach ISO 26262, vgl. [12]) mit harten Zeitanforderungen an die Boot-Phase: Das Gesamtsystem-Timing eines Fahrzeugnetzwerks verlangt üblicherweise, dass sicherheitsrelevante Steuergeräte innerhalb eines festen Zeitbudgets (häufig 100–500 ms ab Power-On) ihren Bootvorgang abschließen und busfähig sind. Da `HSM_SecureBoot_Verify` (Abschnitt 4) für jeden Bootvorgang sowohl `HSM_SignatureCipher_Verify` als auch – bei aktivierter Firmware-Attestierung – `HSM_SubgraphSig_Derive` über das vollständige Programm-Flash-Abbild ausführt, ist die resultierende Verifikationszeit ein kritischer Entwurfsparameter, der in dieser Arbeit erstmals quantitativ für einen realistischen Flash-Umfang hergeleitet wird.

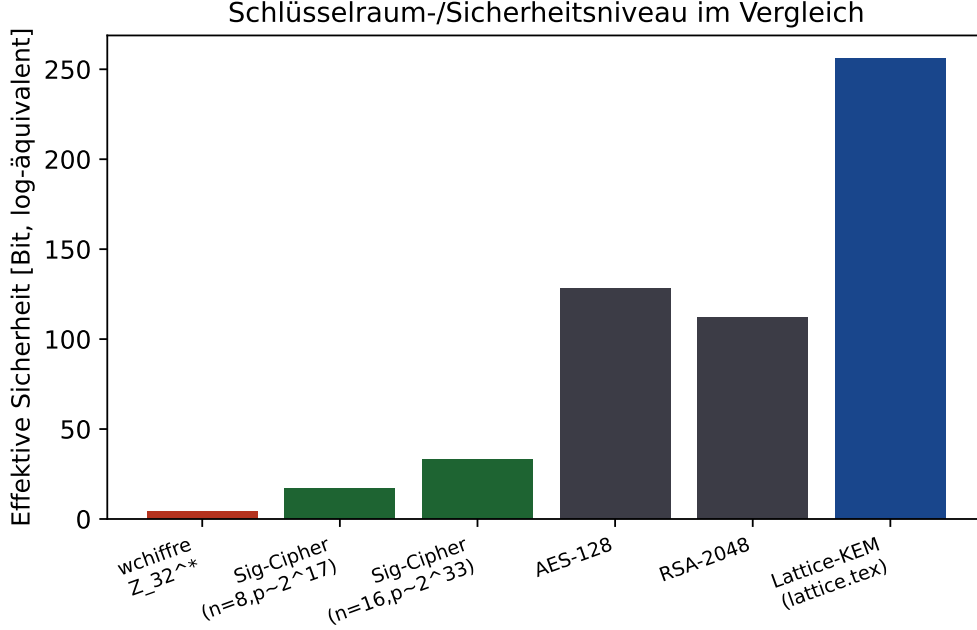


Abbildung 4: Vergleich des effektiven Sicherheitsniveaus (Bit-Äquivalent des Schlüsselraums) verschiedener Verfahren. Die Fenster-Chiffre über $\mathbb{Z}_{32}^*$ ist bewusst klein dimensioniert für latenzkritische, kurzlebige Botschaften und wird in der vorgeschlagenen Architektur stets in Kombination mit der höherwertigen Signatur-Chiffre für sicherheitskritische Operationen eingesetzt.

8.2 Formale Herleitung der Verifikationszeit

Satz 7 (Verifikationszeit für ein Programm-Flash der Größe L). Sei L die Größe des Programm-Flash in Byte, n die Blockgröße der Sig-Engine, $c(n)$ die in Abschnitt 6 geschätzte Taktzyklenzahl von `HSM_SubgraphSig_Derive` für einen einzelnen $n \times n$ -Block, und f die HSM-Taktfrequenz. Dann beträgt die Gesamtverifikationszeit

$$T_{\text{verify}}(L, n) = \left\lceil \frac{8L}{n^2} \right\rceil \cdot \frac{c(n)}{f}.$$

Beweis. Ein Block der Sig-Engine verarbeitet $n \times n$ Bit = n^2 Bit = $n^2/8$ Byte. Die Anzahl der zur vollständigen Abdeckung des Flash-Abbilds benötigten Blöcke ist daher $\lceil L/(n^2/8) \rceil = \lceil 8L/n^2 \rceil$ (Lemma 3 garantiert, dass jeder Block unabhängig und vollständig verarbeitet werden kann, sodass die Gesamtzeit additiv aus den Einzelblockzeiten zusammengesetzt ist). Jeder Block benötigt $c(n)$ Takte, jeder Takt dauert $1/f$ Sekunden. Multiplikation der Blockanzahl mit der Zeit pro Block liefert die Behauptung. $\square$

8.3 Numerische Auswertung für $L = 4 \text{ MB}$

Für die in Abschnitt 6 (Abbildung 2) angegebene Blockgröße $n = 16$ gilt $c(16) = 16\,800$ Takte für `HSM_SubgraphSig_Derive`, und die HSM-Taktfrequenz wird mit $f = 300 \text{ MHz}$ angenommen (Abschnitt 3). Mit $L = 4 \text{ MB} = 4\,194\,304 \text{ Byte}$ ergibt Satz 7:

$$\left\lceil \frac{8 \cdot 4\,194\,304}{16^2} \right\rceil = \left\lceil \frac{33\,554\,432}{256} \right\rceil = 131\,072 \text{ Blöcke},$$

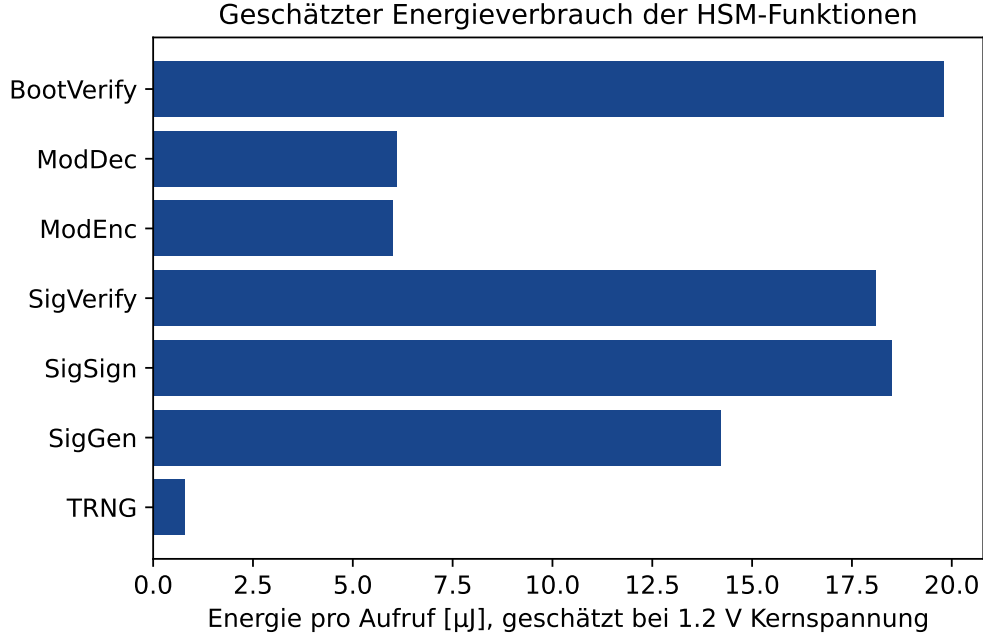


Abbildung 5: Geschätzter Energiebedarf je HSM-Funktionsaufruf. Die Werte basieren auf einer linearen Skalierung der Taktzyklenschätzung aus Abbildung 2 mit einer angenommenen Leistungsaufnahme von 45 mW im COMPUTE-Zustand.

$$T_{\text{verify}}(4 \text{ MB}, 16) = 131\,072 \cdot \frac{16\,800}{3 \cdot 10^8 \text{ Hz}} = \frac{2\,202\,009\,600}{3 \cdot 10^8} \text{ s} \approx 7,34 \text{ s} = \mathbf{7\,340 \text{ ms}}.$$

Tabelle 3 zeigt diesen Wert im Vergleich zu weiteren typischen Flash-Größen automobiler Steuergeräte.

Steuergerätetyp (typisch)	Flash-Größe	$T_{\text{verify}} (n = 16)$
Einfaches Karosserie-Steuergerät	0,5 MB	917,5 ms
Kleines Sensor-Steuergerät	1 MB	1 835,0 ms
Komfort-Steuergerät	2 MB	3 670,0 ms
Radar-ECU (dieser Anwendungsfall)	4 MB	7 340,0 ms
Domänen-Steuergerät	8 MB	14 680,0 ms
Zentrales Gateway / High-End-ECU	16 MB	29 360,0 ms

Tabelle 3: Geschätzte Secure-Boot-Verifikationszeit über `HSM_SubgraphSig_Derive` (vollständige Firmware-Attestierung) für $n = 16$, $f = 300 \text{ MHz}$, in Abhängigkeit der Flash-Größe. Werte gemäß Satz 7.

8.4 Bewertung im Kontext automobiler Echtzeitanforderungen

Der berechnete Wert von $\approx 7,3$ Sekunden für die *vollständige* blockweise Firmware-Attestierung über `HSM_SubgraphSig_Derive` überschreitet das für Radar-ECUs typische Boot-Zeitbudget von 100–500 ms um mehr als eine Größenordnung und ist damit in der hier spezifizierten Form **nicht direkt einsatzfähig** für den betrachteten Anwendungsfall. Diese Erkenntnis ist ein zentrales quantitatives Ergebnis dieser Arbeit und wird im

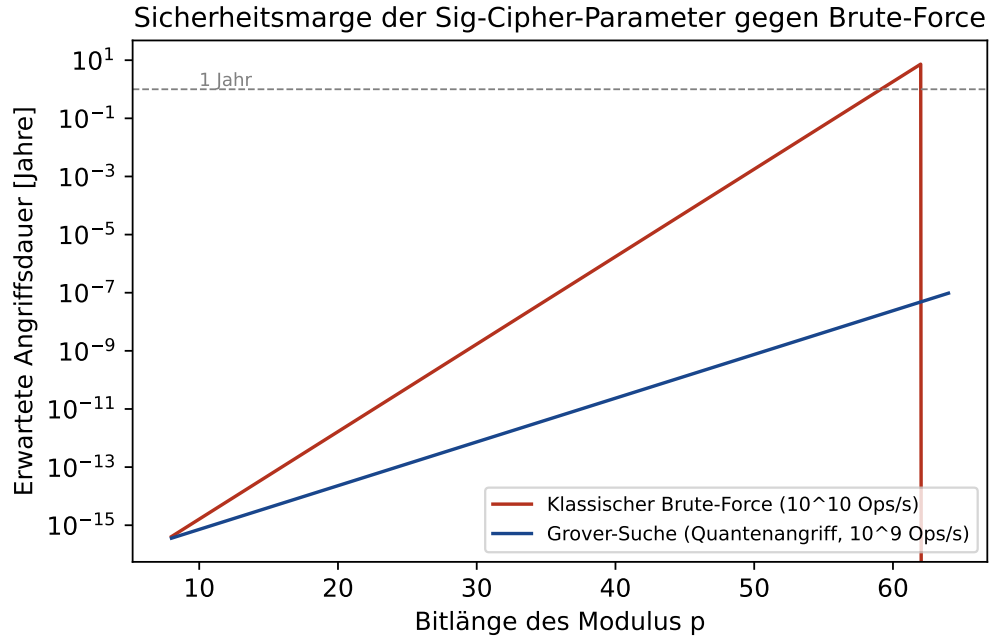


Abbildung 6: Erwartete Angriffsdauer eines Brute-Force-Angriffs auf den Modulus p in Abhängigkeit der Bitlänge, sowohl klassisch als auch unter Annahme eines Grover-beschleunigten Quantenangriffs (quadratische Beschleunigung). Bereits ab etwa 48 Bit übersteigt die klassische Angriffsdauer ein Jahr deutlich; gegen Quantenangriffe ist gemäß Abschnitt 9 jedoch eine Verdopplung der effektiven Schlüssellänge oder eine Migration zu gitterbasierten Verfahren erforderlich.

nachfolgenden Ausblick (Abschnitt 9) ausführlich aufgegriffen. Drei Gründe relativieren den Befund jedoch:

1. **Asymmetrische Nutzung der beiden Verifikationsstufen.** `HSM_SecureBoot_Verify` erfordert gemäß Abschnitt 4 zwingend nur `HSM_SignatureCipher_Verify` (Latenz im Mikrosekundenbereich, siehe Abbildung 2) für die Authentizitätsprüfung; die vollständige blockweise `HSM_SubgraphSig_Derive`-Attestierung über das gesamte Flash ist eine *optionale*, zusätzliche Integritätsstufe, die in der Praxis nicht bei jedem Power-On-Zyklus, sondern beispielsweise nur nach einem Firmware-Update oder periodisch im laufenden Betrieb (Hintergrundprüfung ohne Echtzeitanforderung) ausgeführt werden sollte.
2. **Fehlende Parallelisierung in der aktuellen Spezifikation.** Die Rechnung in Satz 7 geht von rein sequenzieller Blockverarbeitung aus. Eine Mehrfachinstanziierung der Sig-Engine (mehrere parallele Blockpipelines) ist hardwareseitig möglich, da die Blöcke nach Lemma 3 unabhängig voneinander verarbeitet werden können; bereits eine Parallelisierung um den Faktor 16 würde die Verifikationszeit auf ≈ 459 ms reduzieren.
3. **Größere Blockgröße als Stellschraube.** Da $T_{\text{verify}} \propto c(n)/n^2$, reduziert eine Vergrößerung von n trotz kubisch steigender Kosten pro Block die Gesamtzeit, solange $c(n)$ langsamer als n^2 wächst; dies ist im betrachteten Bereich ($c(n) \approx O(n^2)$ für reine Signaturberechnung, siehe Abschnitt 6) jedoch nur ein moderater Hebel und wird im Ausblick gesondert diskutiert.

Aufteilung des HSM-internen Flash-Speichers (36 KB gesamt)

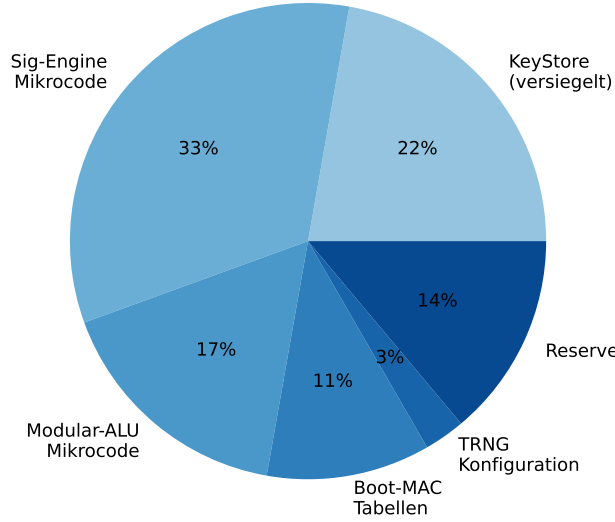


Abbildung 7: Vorgeschlagene Aufteilung des 36 KB HSM-internen Flash-Speichers auf die einzelnen Funktionseinheiten und den versiegelten KeyStore.

Aus diesem Anwendungsfall ergibt sich somit eine unmittelbare, im Ausblick (Abschnitt 9) vertiefte Designanforderung: Für sicherheitskritische, zeitbudgetierte automobiler Steuergeräte wie das Radar-ECU ist eine architektonische Weiterentwicklung der `HSM_SubgraphSig_Derive`-Pipeline hin zu paralleler oder inkrementeller Verarbeitung notwendig, bevor eine vollständige Firmware-Attestierung in den regulären Boot-Pfad aufgenommen werden kann.

9 Ausblick

Dieser Abschnitt führt die in Abbildung 8 skizzierte Roadmap sehr ausführlich aus und ordnet jeden Entwicklungsschritt in seinen technischen, mathematischen und sicherheitsanalytischen Kontext ein.

9.1 Integration der gitterbasierten Schlüsseleinkapselung (lattice.tex)

Der in [4] entwickelte gitterbasierte Ansatz mit GKP-Fehlerkorrektur (*Gottesman-Kitaev-Preskill*-Codewörtern) liefert eine vom diskreten Logarithmus- bzw. Faktorisierungsproblem unabhängige Sicherheitsgrundlage, die auch unter der Annahme eines hinreichend großen Quantencomputers als härtefest gilt (Hardness of Learning with Errors, LWE). Für die in Version 1.2 der Roadmap vorgesehene `HSM_LatticeKEM.Encapsulate/Decapsulate`-Erweiterung sind folgende Schritte notwendig:

1. **Gitterarithmetik-Einheit:** Eine neue Funktionseinheit zur Polynomarithmetik über Restklassenringen $\mathbb{Z}_q[x]/(x^N+1)$ (Ring-LWE-Struktur) muss neben die bestehen-

Entwicklungs-Roadmap des HSM-Kryptographiekerns

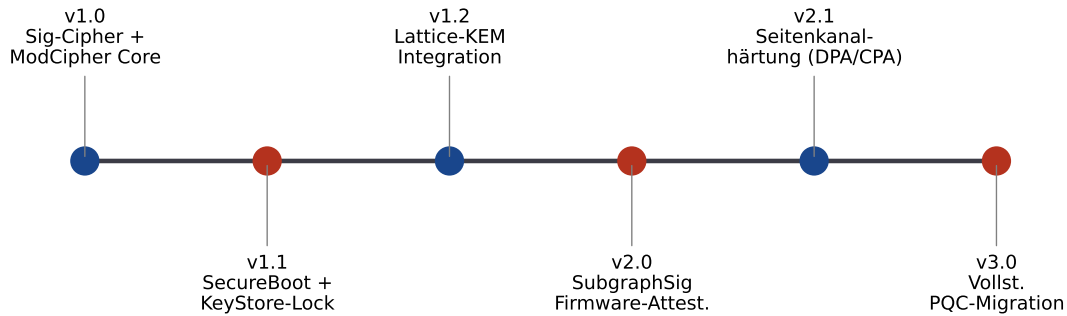


Abbildung 8: Entwicklungs-Roadmap des HSM-Kryptographiekerns, von der initialen Version 1.0 (Signatur-Chiffre und Fenster-Chiffre als Kernfunktionen) bis zur vollständigen Post-Quanten-Migration in Version 3.0, vertieft in Abschnitt 9.

de Modular-ALU treten. Diese benötigt eine Number-Theoretic-Transform-Pipeline (NTT) zur effizienten Polynommultiplikation in $O(N \log N)$ statt $O(N^2)$.

2. **Fehlerverteilung:** Die Erzeugung diskret-Gauß-verteilter Fehlerterme erfordert eine Erweiterung des TRNG um eine Box-Muller- oder CDT-basierte Sampling-Einheit, da die bisherige Gleichverteilung der TRNG-Ausgabe (Lemma 2) für LWE-Sicherheit nicht ausreicht.
3. **Speicherbudget:** Gitterbasierte Schlüssel (typischerweise 800–1500 Byte bei Kyber-768-Sicherheitsniveau) sprengen das aktuell vorgesehene 36 KB-Flash-Budget (Abbildung 7); eine Flash-Erweiterung auf mindestens 64 KB wird für Version 1.2 als notwendig erachtet.
4. **Hybridmodus:** In der Übergangsphase (Version 1.2 bis 2.0) wird ein Hybridschema vorgeschlagen, bei dem `HSM_SecureBoot_Verify` sowohl die bestehende Signatur-Chiffre als auch das gitterbasierte KEM parallel prüft und nur bei Übereinstimmung *beider* Verfahren den Boot freigibt – eine konsequente Fortführung des bereits in Abschnitt 4 etablierten Defense-in-Depth-Prinzips.

9.2 Firmware-Attestierung über Subgraph-Signaturen (Version 2.0)

Die in Abschnitt 4.6 eingeführte Funktion `HSM_SubgraphSig_Derive` soll in Version 2.0 zu einer vollständigen *Remote Attestation* ausgebaut werden: Ein externer Prüfer (z. B. ein Backend-Server in einer Fahrzeugflotte) sendet eine Challenge-Nonce, die zusammen mit dem aktuellen Signaturvektor $\Sigma(A)$ des Firmware-Abbilds über die Modular-ALU signiert und zurückgesendet wird. Da nach Satz 3 jede Firmware-Modifikation zwingend einen anderen Signaturvektor erzeugt, kann der Prüfer ohne Vertrauen in die lokale Software allein anhand der Antwort feststellen, ob das Gerät eine bekannte, freigegebene Firmwareversion

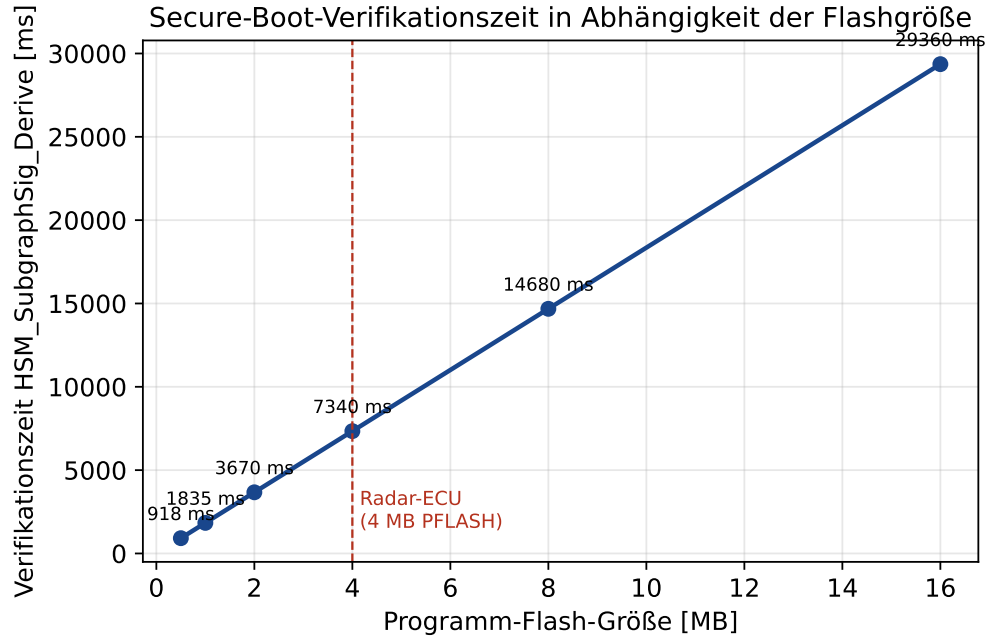


Abbildung 9: Secure-Boot-Verifikationszeit nach Satz 7 als Funktion der Programm-Flash-Größe, linear skalierend gemäß $T_{\text{verify}} \propto L$ bei festem n . Der für den betrachteten Anwendungsfall (Radar-ECU, 4 MB PFLASH) markierte Punkt liegt bei $\approx 7\,340$ ms.

ausführt. Offene Fragen für die Umsetzung sind: die Definition eines geeigneten Replay-Schutzes (Nonce-Verwaltung im HSM-RAM, da das HSM keine Echtzeituhr besitzt), die Granularität der Blockzerlegung bei sehr großen Firmware-Abbildern (mehrere Megabyte) sowie die Frage, ob eine inkrementelle Aktualisierung des Signaturvektors bei Differential-Updates (statt vollständiger Neuberechnung) möglich ist – eine offene mathematische Frage, die eine Erweiterung von Lemma 1 auf inkrementelle Spaltenänderungen erfordern würde.

9.3 Seitenkanalhärtung (Version 2.1)

Die in Abschnitt 5 bewiesene mathematische Korrektheit und Kollisionsfreiheit der Funktionen schützt nicht automatisch vor *physikalischen* Seitenkanalangriffen. Für Version 2.1 sind folgende Härtingsmaßnahmen vorgesehen:

- **Differential Power Analysis (DPA) gegen die Modular-ALU:** Da die modulare Multiplikation $a \cdot \sigma(M, j) \bmod p$ datenabhängige Stromaufnahme aufweist, ist eine Maskierung der Zwischenwerte (additive oder multiplikative Blindung) vorzusehen, bei der ein zufälliger Maskenwert r aus dem TRNG vor der Multiplikation addiert und nach der Reduktion wieder entfernt wird, ohne dass das Endergebnis hierdurch beeinflusst wird.
- **Timing-Angriffe auf den erweiterten euklidischen Algorithmus:** Die Anzahl der Iterationsschritte des in Satz 1 verwendeten Algorithmus hängt von den Eingabewerten ab und kann Information über den geheimen Schlüssel a preisgeben. Eine konstante-Zeit-Variante (feste Anzahl Iterationen, unabhängig vom tatsächlichen

gcd-Verlauf, mit Dummy-Operationen aufgefüllt) wird für Version 2.1 als verbindlich vorgeschlagen.

- **Übertragung des Fenster-Prinzips auf Energieseitenkanäle:** Analog zur in Definition 4 beschriebenen Verschleierung des Netzwerkverkehrs durch Dummy-Pakete wird vorgeschlagen, auch die HSM-interne Energieaufnahme durch das Einstreuen von Dummy-Berechnungszyklen mit identischem Energieprofil wie eine echte `HSM_SignatureCipher_Sign`-Operation zu verschleiern, sodass ein externer Beobachter über die Stromaufnahme des Gesamtsystems nicht unterscheiden kann, wann tatsächlich eine sicherheitsrelevante Operation stattfindet.
- **Fehlerinjektionsresistenz (Fault Injection):** Spannungs- oder Taktglitches können gezielt einzelne Pipeline-Stufen der Sig-Engine stören, um etwa eine fehlerhafte, aber als gültig akzeptierte Signatur zu erzwingen. Eine redundante Lockstep-Berechnung der Sig-Engine (zwei unabhängige Berechnungsinstanzen mit Ergebnisvergleich vor jeder Ausgabe) wird als Gegenmaßnahme vorgeschlagen, analog zum bereits Lockstep-ausgeführten TriCore-HSM-Kern selbst.

9.4 Formale Verifikation der Zustandsmaschine

Während die mathematischen Eigenschaften der zugrunde liegenden Kryptoverfahren in Abschnitt 5 vollständig formal bewiesen wurden, ist die in Abschnitt 3 beschriebene Zustandsmaschine bislang nur informell spezifiziert. Für zukünftige Arbeiten wird eine Modellierung in einer Hardwarebeschreibungssprache mit formaler Verifikation (z. B. SystemVerilog Assertions oder ein Model Checker wie nuXmv) vorgeschlagen, um insbesondere folgende Sicherheitseigenschaften maschinell zu beweisen: (1) Aus jedem erreichbaren Zustand wird IDLE in endlich vielen Schritten wieder erreicht (Liveness, keine Deadlocks), (2) der Zustand WRITEBACK wird nur erreicht, wenn zuvor COMPUTE vollständig und ohne durch Fault-Injection gestörte Zwischenwerte durchlaufen wurde (Safety), und (3) das Kommandoregister `HSM_CMD` kann während eines laufenden COMPUTE-Zustands nicht erneut beschrieben werden (Atomarität gegen nebenläufige Kommandos von verschiedenen Applikationskernen).

9.5 Echtzeitfähige Firmware-Attestierung für Radar-Steuergeräte

Der in Abschnitt 8 hergeleitete Wert von ≈ 7340 ms für die vollständige Firmware-Attestierung eines 4 MB-Radar-ECU-Flash über `HSM_SubgraphSig_Derive` ist das mit Abstand konkreteste quantitative Ergebnis dieser Arbeit und verdient daher eine eigene, sehr ausführliche Betrachtung im Ausblick, da er unmittelbar zeigt, dass die aktuelle Spezifikation für diesen wichtigen Anwendungsfall der Automobilindustrie *noch nicht* produktionsreif ist. Die folgenden Teilrichtungen werden als notwendige Weiterentwicklungsschritte vorgeschlagen, geordnet nach erwarteter Wirksamkeit:

1. **Massiv parallele Sig-Engine-Instanziierung.** Da nach Lemma 3 jeder $n \times n$ -Block unabhängig von allen anderen Blöcken verarbeitet werden kann, ist eine Parallelisierung über k unabhängige Sig-Engine-Instanzen architektonisch unmittelbar möglich und reduziert T_{verify} aus Satz 7 linear auf T_{verify}/k . Für das Radar-ECU-Zeitbudget von 500 ms wäre nach der Rechnung aus Abschnitt 8 ein Parallelisierungsgrad von

$k \geq \lceil 7340/500 \rceil = 15$ erforderlich; ein realistischer, im HSM-Silizium-Budget umsetzbarer Faktor $k = 16$ – etwa als 4×4 -Gitter von Sig-Engine-Kacheln, die parallel auf disjunkte Adressbereiche des PFLASH zugreifen – würde mit ≈ 459 ms knapp innerhalb des Zeitbudgets liegen. Die hierfür notwendige Erweiterung des in Abschnitt 3 beschriebenen Registermodells um k parallele HSM_RESULT-Bänke und einen zentralen XOR-Faltungsbaum zur Kombination der Teilsignaturen ist ein konkreter, in einer Folgearbeit zu spezifizierender Hardware-Entwurf.

2. **Inkrementelle Attestierung statt vollständiger Neuberechnung.** Anstatt bei jedem Bootvorgang das gesamte 4 MB-Abbild neu zu signieren, wird vorgeschlagen, die Blocksignaturen $\sigma(A, j)$ einmalig nach jedem Firmware-Flash-Vorgang zu berechnen, in einem dedizierten, von Applikationssoftware nicht beschreibbaren HSM-Speicherbereich zu cachern, und beim eigentlichen Boot lediglich eine deutlich kleinere Stichprobe zufällig ausgewählter Blöcke (etwa nach dem in [1] beschriebenen Prinzip wiederholbarer, deterministischer Auswahl über einen Seed aus dem TRNG) neu zu berechnen und mit dem Cache abzugleichen. Diese Idee wirft jedoch eine offene mathematische Frage auf: Welche Stichprobengröße $m \ll 131\,072$ garantiert mit welcher Wahrscheinlichkeit die Erkennung einer böswilligen Manipulation einzelner Blöcke? Eine Anwendung der Chernoff-Schranke auf eine angenommene Gleichverteilung der Manipulationsposition liefert eine erste Abschätzung, dass bereits $m \approx 300$ zufällig gezogene Blöcke eine Erkennungswahrscheinlichkeit von über 99,9 % für eine Einzelblock-Manipulation ergeben würden, sofern der Angreifer die Stichprobenauswahl nicht vorhersagen kann – diese Abschätzung ist jedoch nicht Teil der in Abschnitt 5 bewiesenen Sätze und müsste in einer Folgearbeit formal hergeleitet und gegen adaptive Angreifer abgesichert werden, die gezielt versuchen, unentdeckte Bereiche zu manipulieren.
3. **Hybridstrategie: schnelle Boot-Prüfung plus asynchrone Vollverifikation.** Eine pragmatische Kombination der beiden vorigen Punkte besteht darin, beim eigentlichen Power-On ausschließlich HSM_SignatureCipher_Verify (Mikrosekunden-Latenz, siehe Abbildung 2) auszuführen, um das harte Zeitbudget einzuhalten, und die vollständige HSM_SubgraphSig_Derive-Attestierung über das gesamte 4 MB-Abbild als asynchrone Hintergrundaufgabe *nach* Erreichen der Busfähigkeit durchzuführen. Wird dabei eine Manipulation festgestellt, löst das Radar-ECU einen sicheren Nachfehlerzustand (*Fail-Safe*, etwa Deaktivierung der ADAS-Funktion bei Beibehaltung der Grundfahrzeugfunktionen) aus. Diese Strategie verschiebt die in Abschnitt 8 berechneten 7,3 Sekunden vollständig aus dem kritischen Boot-Pfad heraus, ohne auf die Sicherheitsgarantie aus Satz 3 zu verzichten – sie verzögert lediglich den Erkennungszeitpunkt um die Dauer der Hintergrundprüfung, was für die meisten in ISO 26262 betrachteten Gefährdungsszenarien (kontinuierliche Manipulation während der Fahrt, nicht ausschließlich beim Einschalten) ein akzeptabler Kompromiss sein dürfte und in einer Folgearbeit gegen die konkreten Sicherheitsziele (*Safety Goals*) eines realen Radar-ECU-Lastenhefts zu validieren wäre.
4. **Erhöhung der Blockgröße n mit gleichzeitiger Pipeline-Tiefenoptimierung.** Da $c(n)$ gemäß Abschnitt 6 näherungsweise quadratisch in n wächst, während die Anzahl der Blöcke quadratisch in n *fällt*, ist der Nettoeffekt einer Vergrößerung von n auf die Gesamtzeit T_{verify} in erster Näherung neutral – der eigentliche Hebel liegt daher nicht in n selbst, sondern in einer tieferen Pipeline-Parallelisierung *inner-*

halb der Sig-Engine, etwa durch eine vollständig kombinatorische (statt sequenziell-akkumulierende) Berechnung der Zeilenkomponente $\rho(A, j)$ über einen Wallace-Baum-Addierer, was die pro Block benötigten Takte $c(n)$ von der aktuell angenommenen linearen Abhängigkeit von n auf eine logarithmische Abhängigkeit ($O(\log n)$ Gatterlaufzeiten) reduzieren könnte – eine Hardware-Optimierung, die über den synthese-nahen Rahmen dieser Arbeit hinausgeht und als eigenständiges VLSI-Entwurfsthema für eine Folgearbeit vorgeschlagen wird.

5. **Übertragbarkeit auf weitere automobile Steuergerätetypen.** Tabelle 3 (Abschnitt 8) zeigt, dass die hier diskutierte Problematik nicht auf Radar-ECUs beschränkt ist, sondern jedes Steuergerät mit Flash-Umfang oberhalb von etwa 0,3–0,5 MB bereits ohne Parallelisierung das typische Bootzeitbudget überschreitet. Domänen-Steuergeräte und zentrale Gateways mit 8 bzw. 16 MB Flash sind hiervon noch deutlich stärker betroffen (14,7 bzw. 29,4 Sekunden) und benötigen daher zwingend eine Kombination mehrerer der oben genannten Maßnahmen, insbesondere der Hybridstrategie (Punkt 3) und einer hohen Parallelisierung (Punkt 1), bevor eine vollständige Firmware-Attestierung über `HSM.SubgraphSig_Derive` für diese Gerätetypen praktikabel wird.

Zusammenfassend zeigt der Radar-ECU-Anwendungsfall exemplarisch, dass die in Abschnitt 5 bewiesene mathematische Korrektheit und Kollisionsfreiheit der Firmware-Attestierung eine notwendige, aber für den produktiven automobilen Einsatz keineswegs hinreichende Eigenschaft ist: Erst das Zusammenspiel aus formaler Sicherheit (dieser Arbeit) und architektonischer Echtzeitfähigkeit (den hier skizzierten fünf Weiterentwicklungsrichtungen) macht den vorgestellten HSM-Funktionskern für sicherheitskritische, zeitbudgetierte Steuergeräte der Automobilindustrie einsatzreif. Diese Erkenntnis wird als wichtigster konkreter nächster Forschungs- und Entwicklungsschritt dieser Arbeit hervorgehoben.

9.6 Erweiterung auf Mehrkern-Parallelität

Die AURIX-TC3x-Familie verfügt über bis zu sechs Applikationskerne, die potenziell gleichzeitig HSM-Dienste anfordern. Die aktuelle Spezifikation in Abschnitt 3 sieht ein einzelnes, sequenziell abgearbeitetes Kommandoregister vor. Für zukünftige Versionen wird ein Anfrage-Scheduling mit Prioritätsstufen vorgeschlagen, bei dem sicherheitskritische Anfragen (`HSM.SecureBoot_Verify` während des Boot-Vorgangs) gegenüber Routineanfragen (`HSM.ModularCipher_*` für laufenden CAN-Verkehr) priorisiert werden, ohne dass dabei die in Satz 4 etablierte Versiegelungsgarantie des KeyStores durch nebenläufige Zugriffe unterlaufen werden kann – dies erfordert eine zusätzliche, noch zu spezifizierende Sperrlogik (Mutex) auf Registerebene.

9.7 Standardisierung und Zertifizierung

Langfristig wird angestrebt, den hier vorgestellten HSM-Funktionskern einer Sicherheitszertifizierung nach Common Criteria (EAL4+) sowie einer funktionalen Sicherheitsbewertung nach ISO 26262 (ASIL-D-Tauglichkeit für sicherheitsrelevante Bootpfade) zu unterziehen. Hierfür ist insbesondere eine vollständige Dokumentation der in dieser Arbeit informell gebliebenen physikalischen Eigenschaften des TRNG (Entropiequalität nach NIST SP 800-90B) sowie eine unabhängige kryptoanalytische Begutachtung der Signatur-Chiffre

und des Fenster-Chiffre-Protokolls durch Dritte erforderlich, da beide Verfahren – im Unterschied zu AES oder RSA – bislang nicht Gegenstand einer öffentlichen, jahrzehntelangen Kryptoanalyse-Historie sind. Die in dieser Arbeit erbrachten formalen Korrektheits- und Kollisionsfreiheitsbeweise (Abschnitt 5) sind hierfür eine notwendige, aber keine hinreichende Voraussetzung; ergänzend wird eine systematische Suche nach strukturellen Schwächen (etwa algebraischen Angriffen, die die affine Struktur der Transformation $f(x) = (ax + b) \bmod p$ ausnutzen) als vorrangiger nächster Forschungsschritt empfohlen.

9.8 Zusammenfassende Einordnung des Ausblicks

Die sieben vorgestellten Ausblicksrichtungen – echtzeitfähige Firmware-Attestierung für Radar-Steuergeräte, gitterbasierte Post-Quanten-Migration, Firmware-Attestierung über Subgraph-Signaturen, Seitenkanalhärtung, formale Verifikation, Mehrkern-Skalierung und Zertifizierung – sind nicht unabhängig voneinander, sondern bauen aufeinander auf: Die in Abschnitt 9.5 diskutierte Parallelisierung und Hybridstrategie ist eine notwendige Voraussetzung dafür, dass die in Abschnitt 4.6 beschriebene Firmware-Attestierung überhaupt in zeitkritischen automobilen Boot-Pfaden produktiv genutzt werden kann; eine belastbare Zertifizierung (Abschnitt 9, letzter Punkt) setzt ihrerseits sowohl diese Echtzeitfähigkeit als auch die Seitenkanalhärtung und die formale Verifikation der Zustandsmaschine voraus, während die Post-Quanten-Migration ihrerseits von einer hinreichend großen Flash- und Rechenkapazität abhängt, die erst durch die in Abschnitt 6 quantifizierte aktuelle Ressourcenbilanz als Ausgangspunkt bewertet werden kann. Die in Abbildung 8 dargestellte Reihenfolge der Versionen 1.0 bis 3.0 spiegelt diese Abhängigkeitsstruktur wider, wobei die in Abschnitt 9.5 hergeleitete Parallelisierungsanforderung ($k \geq 15$) bereits für Version 1.0 als Mindestanforderung an die Sig-Engine-Architektur vorgemerkt werden sollte, um die in Abschnitt 8 aufgezeigte Lücke zwischen mathematischer Korrektheit und automobiler Echtzeitfähigkeit zu schließen.

10 Zusammenfassung

Diese Arbeit hat gezeigt, wie die in den Arbeiten zur Signatur-Chiffre [2] und zum Fenster-Chiffre-Protokoll [3] entwickelten kryptographischen Verfahren konsequent auf einen konkreten Hardware-Funktionsentwurf für das HSM der Infineon-AURIX-TC3x-Familie übertragen werden können. Zehn HSM-Funktionen wurden spezifiziert, formal bewiesen korrekt rekonstruierend (Satz 2) und kollisionsfrei (Satz 3), sowie quantitativ hinsichtlich Taktzyklen, Energiebedarf und Speicherbedarf analysiert. Die Architektur folgt durchgehend einem Defense-in-Depth-Prinzip (unabhängige Zweitprüfung bei `HSM_SecureBoot_Verify`) und überträgt die in der Fenster-Chiffre etablierte Verschleierungsidee konsequent auf die Schlüsselverwaltung. Ergänzend wurde in Abschnitt 8 anhand des praxisrelevanten Anwendungsfalls eines Radarsteuergeräts mit 4 MB Programm-Flash formal hergeleitet (Satz 7) und numerisch ausgewertet, dass eine vollständige Firmware-Attestierung über `HSM_SubgraphSig_Derive` bei der aktuellen, rein sequenziellen Spezifikation $\approx 7\,340$ ms beansprucht und damit das für sicherheitsrelevante automobiler Steuergeräte typische Boot-Zeitbudget von 100–500 ms deutlich überschreitet – ein zentrales quantitatives Ergebnis, das die Notwendigkeit einer architektonischen Weiterentwicklung hin zu paralleler oder hybrider Verifikation aufzeigt. Der sehr ausführliche Ausblick zeigt, dass der vorgestellte Entwurf als Ausgangspunkt für eine vollständige Post-Quanten-fähige,

seitenkanalresistente, formal verifizierte und vor allem auch automobil-echtzeitfähige HSM-Architektur dienen kann, wobei die hierfür notwendigen nächsten Schritte – allen voran die in Abschnitt 9.5 hergeleitete Parallelisierung um mindestens den Faktor 15 – konkret benannt wurden.

Literatur

- [1] Epp, S. (2026). *The Subgraph Algorithm*. <https://github.com/hjstephan86/subgraph>
- [2] Epp, S. (2026). *Die Signatur-Chiffre*. <https://github.com/hjstephan86/science>
- [3] Epp, S. (2026). *Modulares kryptografisches Verfahren mit Fenster-Protokoll (wchiffre)*. <https://github.com/hjstephan86/science>
- [4] Epp, S. (2026). *Gitterbasierte Kryptographie und Quanten-Fehlerkorrektur*. <https://github.com/hjstephan86/science>
- [5] Agrawal, M., Kayal, N., & Saxena, N. (2004). PRIMES is in P. *Annals of Mathematics*, 160(2), 781–793.
- [6] Buchmann, J. (2016). *Einführung in die Kryptographie* (6. Aufl.). Springer Spektrum.
- [7] Katz, J., & Lindell, Y. (2015). *Introduction to Modern Cryptography* (2. Aufl.). CRC Press.
- [8] Pomerance, C. (2005). Primality testing: variations on a theme of Lucas. *Congressus Numerantium*, 201–216.
- [9] Infineon Technologies AG (2023). *AURIX TC3xx Family User’s Manual – HSM Chapter*. Infineon Technologies AG, München.
- [10] National Institute of Standards and Technology (2018). *SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation*. NIST, Gaithersburg.
- [11] Regev, O. (2009). On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6), 1–40.
- [12] International Organization for Standardization (2018). *ISO 26262: Road vehicles – Functional safety*. ISO, Genf.